

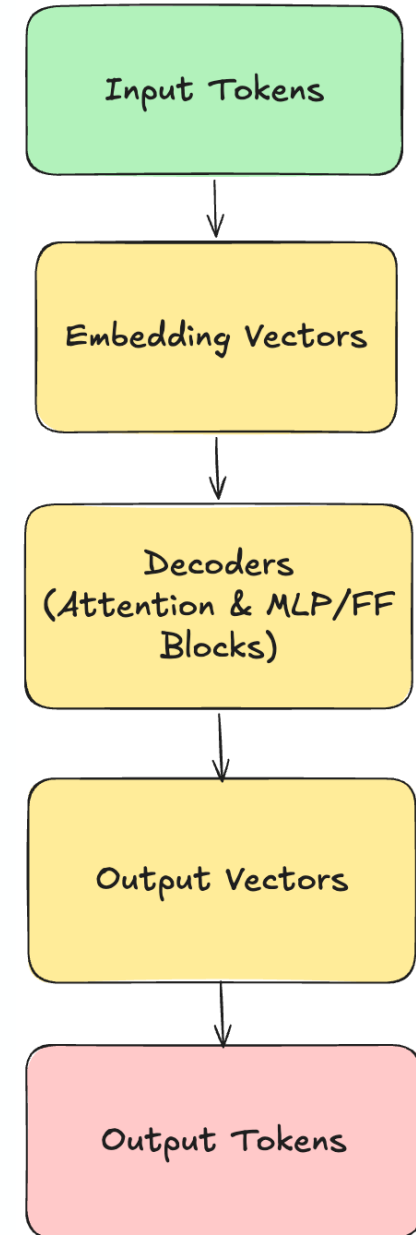
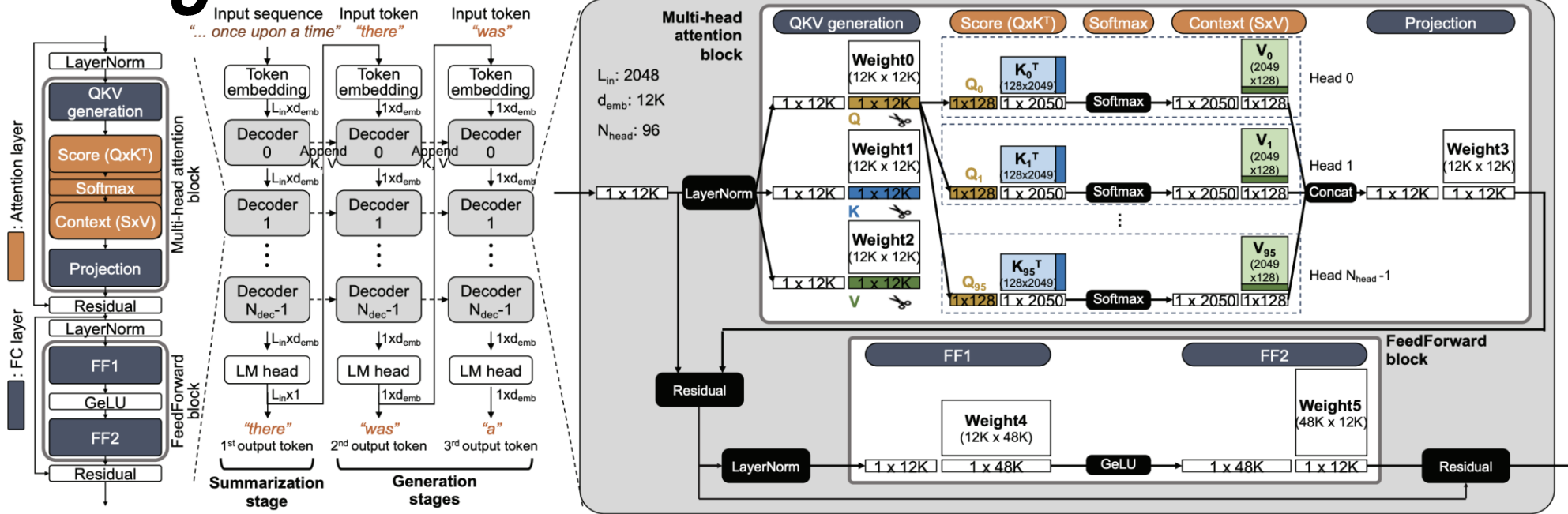
KV-PIMple

PIM-aware KV-cache Allocation for LLM Decode

Kiet Vu

9/15/26

Background:



1. Prefill: The attention layers produce K (key) Q (query) V (value), with KV containing the context of previous tokens
2. Decode: scores Q against all past K $\rightarrow$ weights $\rightarrow$ weighted sum of V
3. KV cache: instead of recalculating the 2 matrices again per output token $\rightarrow$ store them $\rightarrow$ main usage of GPU memory (excluding weights) $\rightarrow$ trade compute-bound for memory-bound (wide margin)

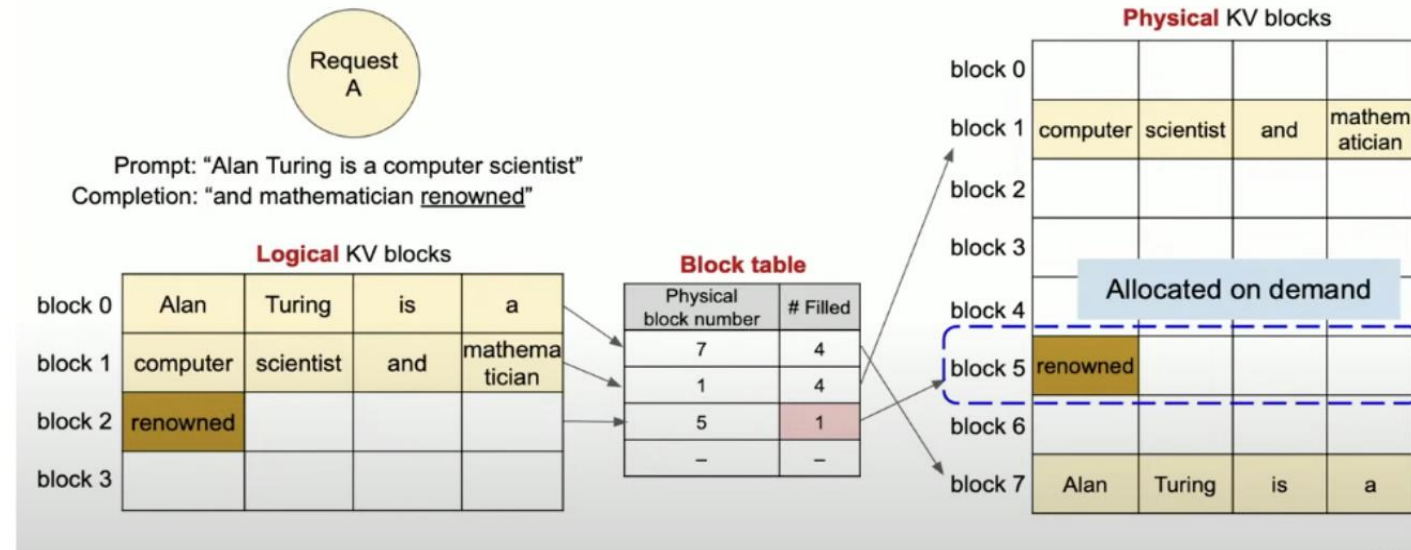
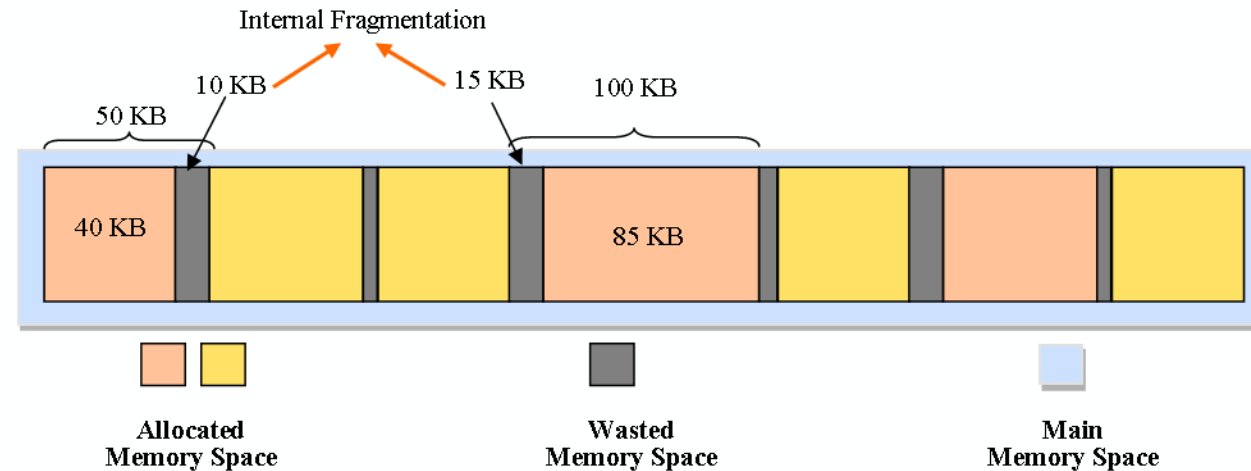
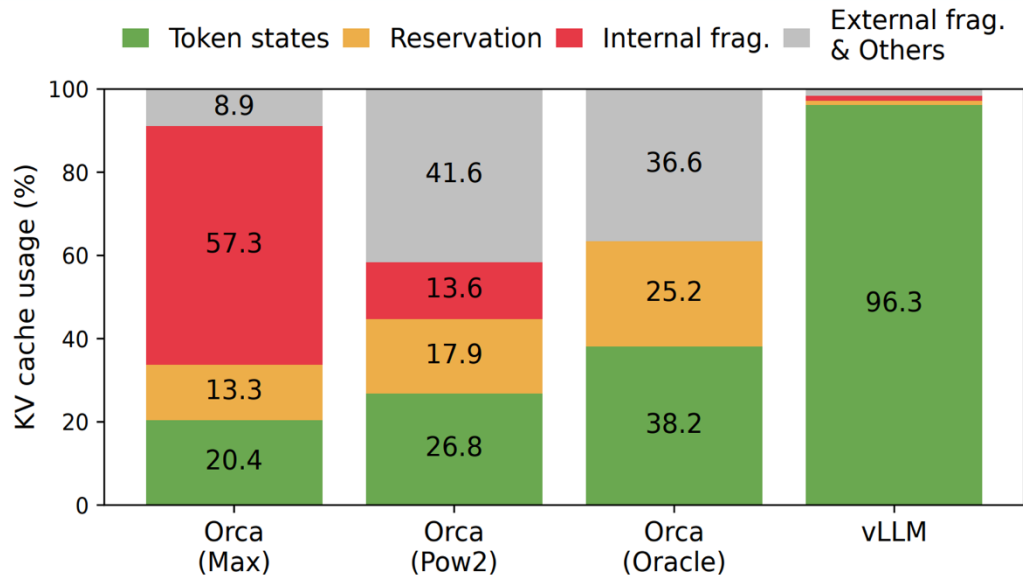
Problem: The cache grows unpredictably since output length is unknown & memory fragmentation $\rightarrow$ reserve & waste memory space

Background:

Problem: Memory Fragmentation

Solution: vLLM & PagedAttention

- PagedAttention: Borrows virtual memory & paging from OS memory management
- Prefix sharing: identical prefixes point at the same physical block, copy-on-write on divergence



Background:

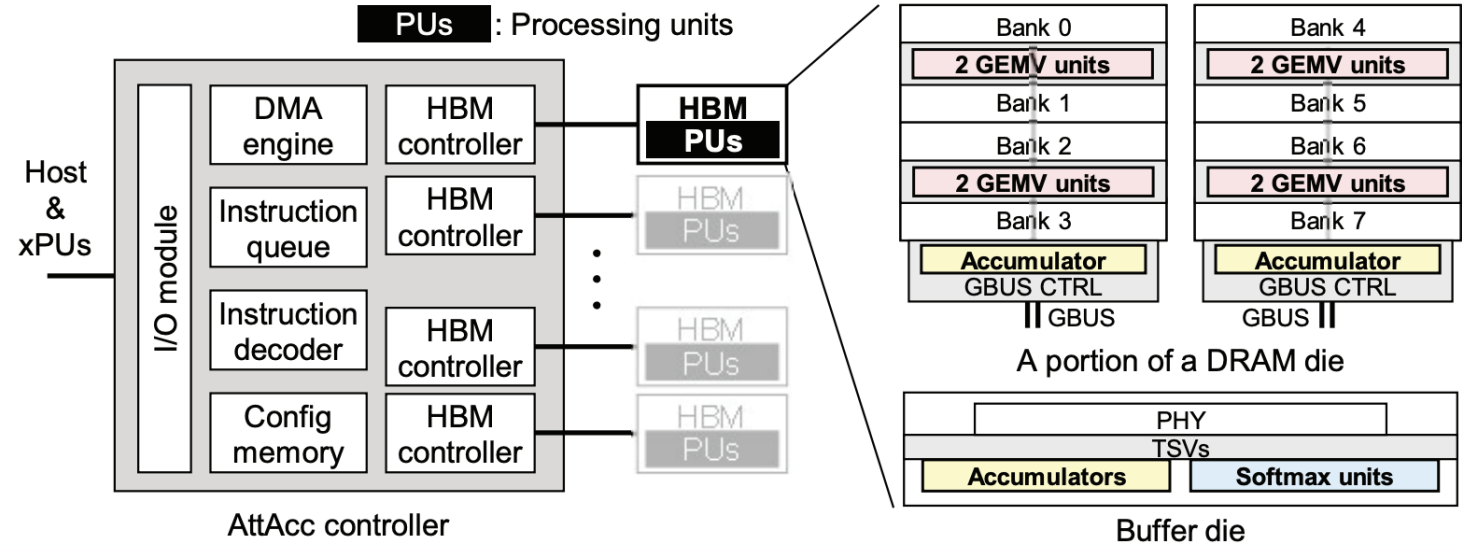
Persisting Problem: Memory Bandwidth

Solution: processing-in-memory (PIM) -> take advantage of the internal bandwidth by doing Attention inside the banks

- All bank ops: activate the *same row index* in all banks of a channel, then broadcast one command -> bank parallelism

- However, this only works if the data sits at matching row indices across banks (row group), which most PIM papers assume

- **Gap:** Most papers just assume the layout



NeuPIMs perform this operation using the PIM_GWRITE command, which copies a specific row of a particular bank to the global vector buffer. Similarly, it needs another command to activate the rows of the operand matrix into PIM row buffers across the banks. For this, the host produces grouped activation commands, called “PIM_ACTIVATION”, which activate PIM row buffers of multiple banks simultaneously, usually for 4 banks at a time due to the power constraints

NeuPIMs | ASPLOS '24

All Bank Mode: In contrast to the standard SB mode, we expose the on-chip bandwidth of DRAM to PIM execution units by allowing concurrent accesses to multiple banks with a single DRAM command (Fig. 3(b)). As such, we implement all-bank control logic to support all-bank operation mode (AB mode), where the BA and BG of a given column address are ignored and the same row and column of all the banks are concurrently accessed in a lock-step manner by a single DRAM command. In AB mode, PIM-HBM with 16 banks

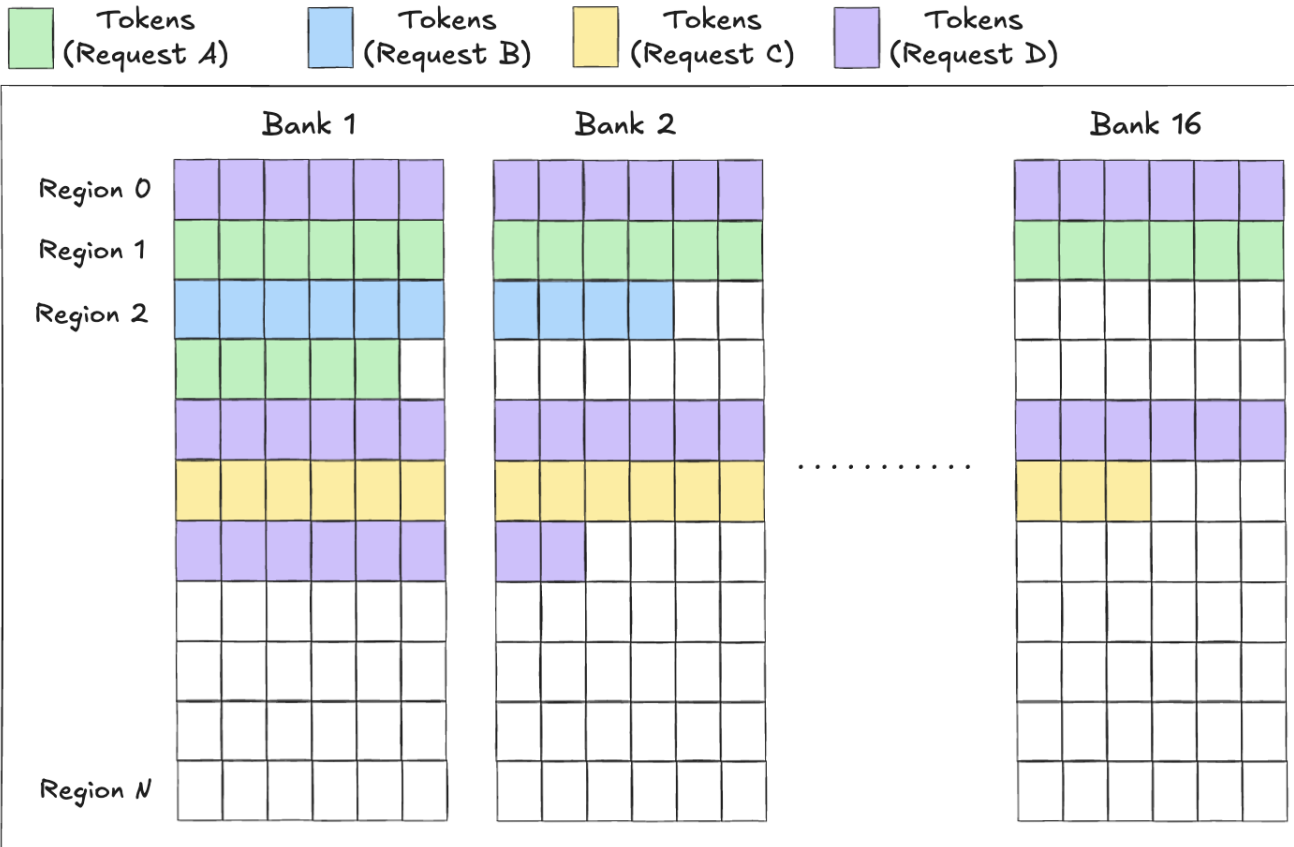
Samsung HBM2-PIM & Aquabolt-XL | Hot Chips 33

PIM commands issued to the HBM include the ones for metadata setup and data movement. PIM_SET_CONFIG writes the partitioning information of the KV matrices to the GEMV units. PIM_ACT_AB performs row activation of the same address in all banks, similar to prior works [36, 37]. PIM_MAC_AB

AttAcc! | ASPLOS '24

KV-PIMple: PIM-aware KV-cache Allocation

The Fix

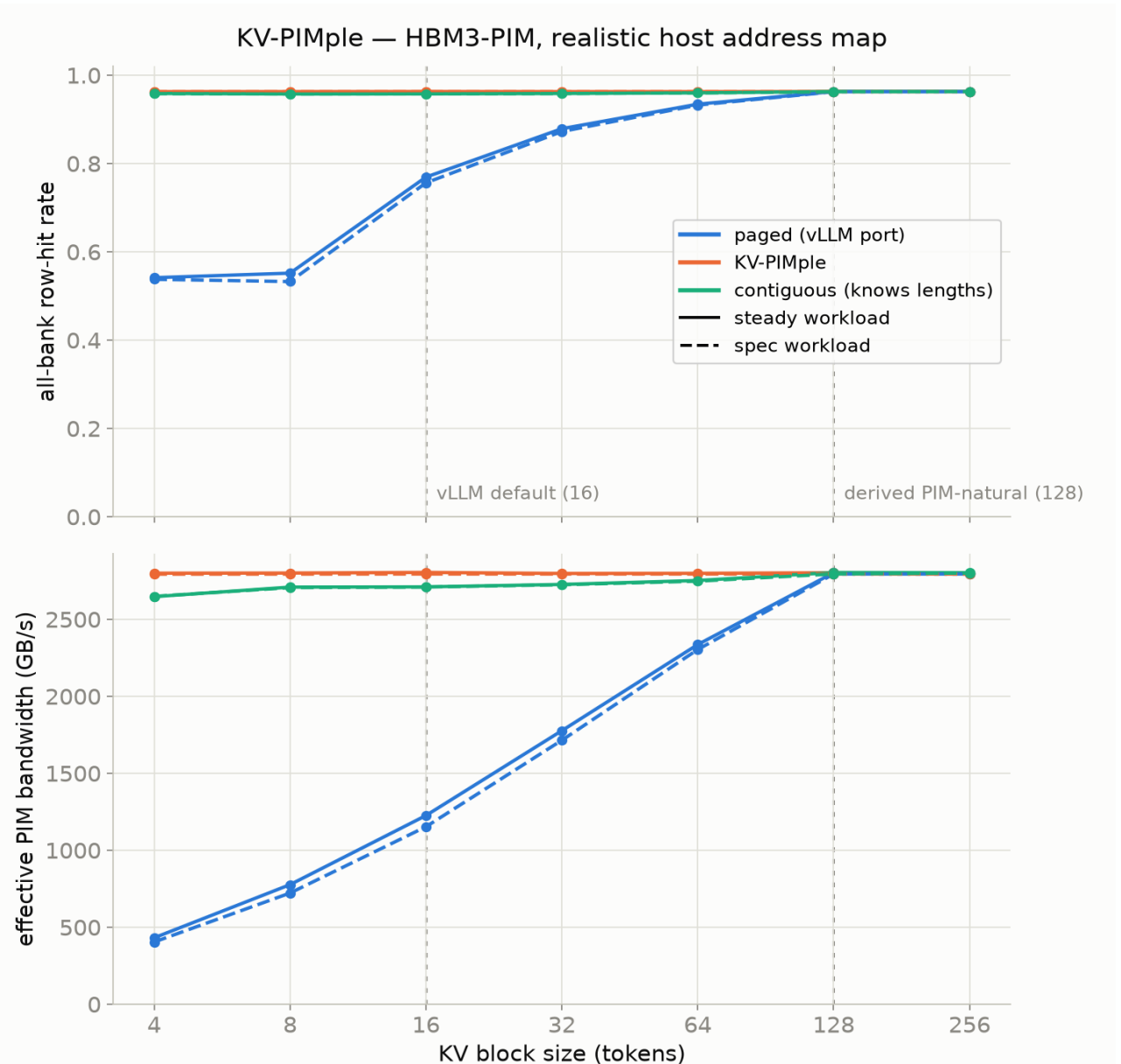


- Bounded contiguity: Turn the pool into regions, which holds $\#_blocks$ that fills one row-group across all channels
- Fill a region in order, claim another only when it's full. A sequence's regions can be scattered
- $region_size = row_size \times banks \times channels$
 $= 1\text{ KB} \times 16 \times 32 = 512\text{ KB}$
- $token_size = KV_matrices \times n_kv_heads \times head_dim \times precision$
 $\times 32 = 2 \times 8 \times 128 \times 2\text{ (fp16)} = 4096\text{ B} = 4\text{ KB/layer}$
- $\#_tokens_per_region = 512\text{ KB} / 4\text{ KB} = 128\text{ tokens}$
- $\#_blocks_per_region = 128\text{ tokens} / 16\text{ tokens} = 8\text{ blocks}$
- When no free region is available, we fall back to filling blocks to partial regions -> degrades to baseline behavior
- Runtime allocation: $\#_blocks_per_region$ derived at startup from DRAM geometry and model shape
- We pick the lowest free slot from that sequence's region heap, and the lowest empty region when it fills
- $row = \text{floor}(blockID / \#_blocks_per_region)$

KV-PIMple: PIM-aware KV-cache Allocation

Measurements

- At the standard 16-token block, KV-PIMple raises the row-hit rate from **0.77** to **0.96** and effective bandwidth from **1226** to **2805 GB/s** -> **2.3x**
- At 4-token blocks the gap widens to **6.5x**. Under speculative decoding it is **2.4x**
- However, at 128 tokens every allocator scores the same since a 128-token block is exactly one aligned region
- Nevertheless, using 128-token blocks gives up the fine-grained packing of paging. With 16-token blocks and PIM-aware allocator we keep the packing and get the same result

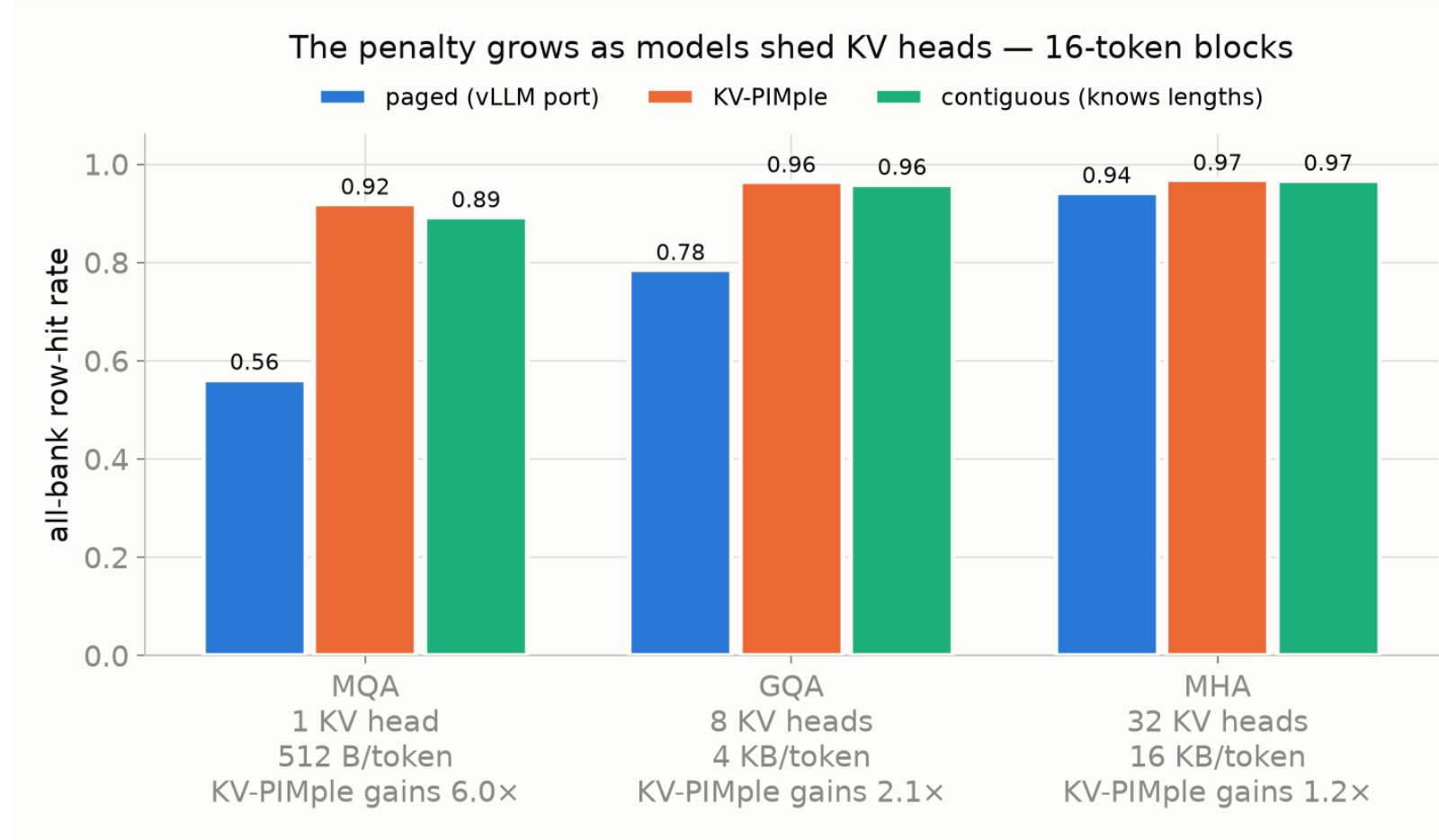


KV-PIMple: PIM-aware KV-cache Allocation

Model Configs

Token Size & Model Heads:

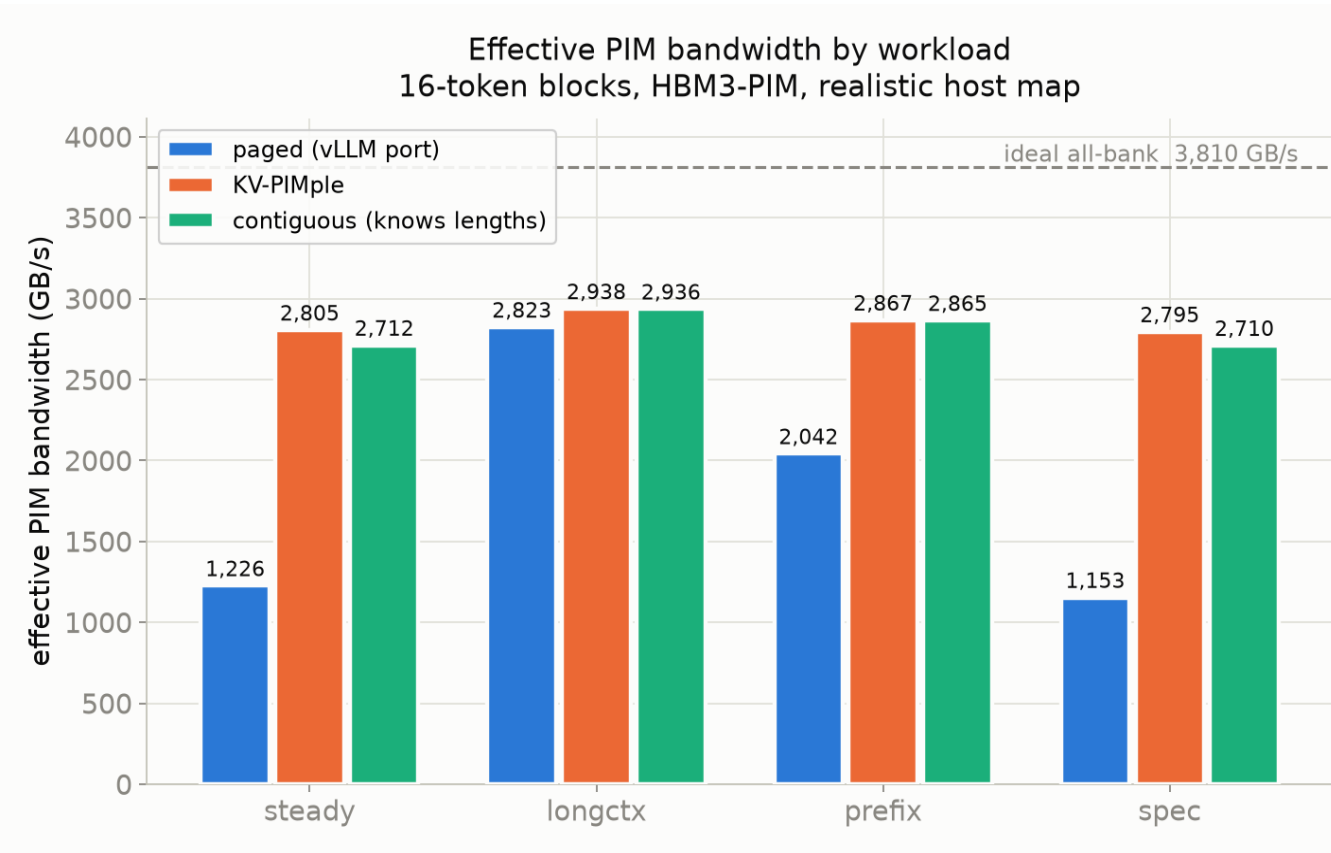
- Token size increases proportionally with the number of heads
- As token size increases, block size (bytes) also increases, occupying more rows across banks → filling regions → row-hit rate increases
- However, since memory storage is a constraint, we can't just increase everything naively



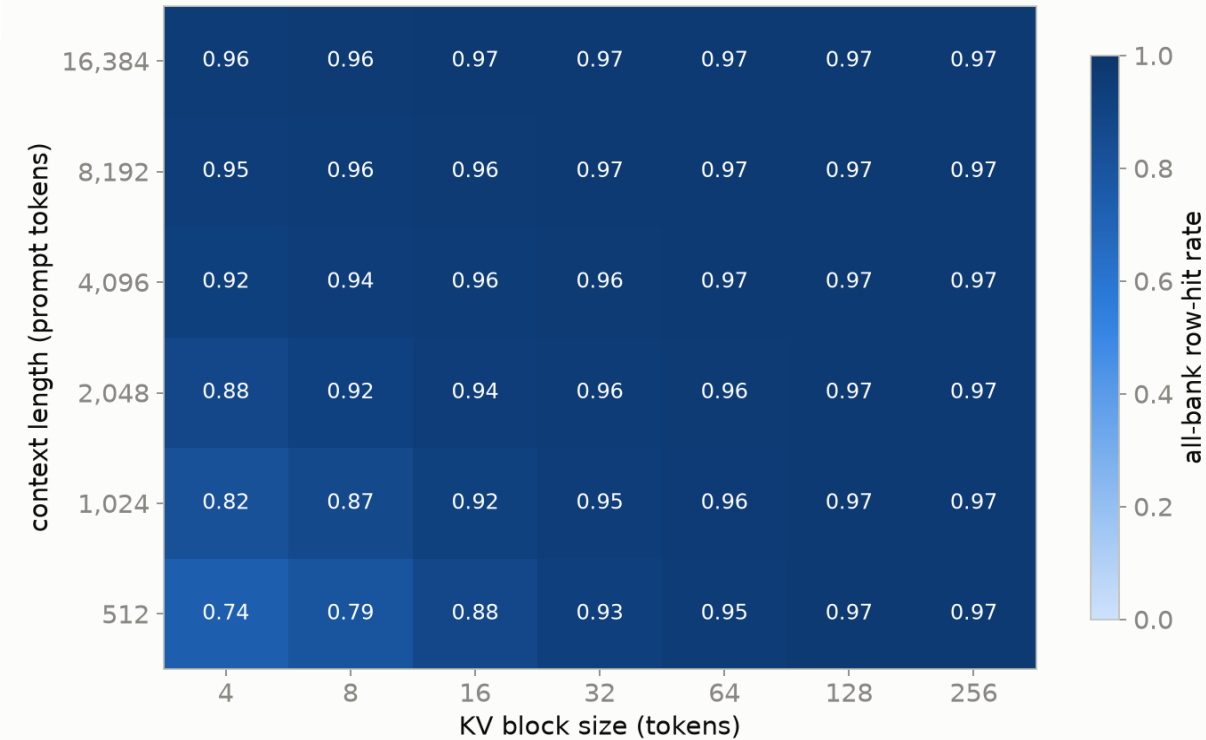
**The gains here are PIM bandwidth*

KV-PIMple: PIM-aware KV-cache Allocation

Workload



All-bank row-hit rate — paged allocator, HBM3-PIM, realistic host map



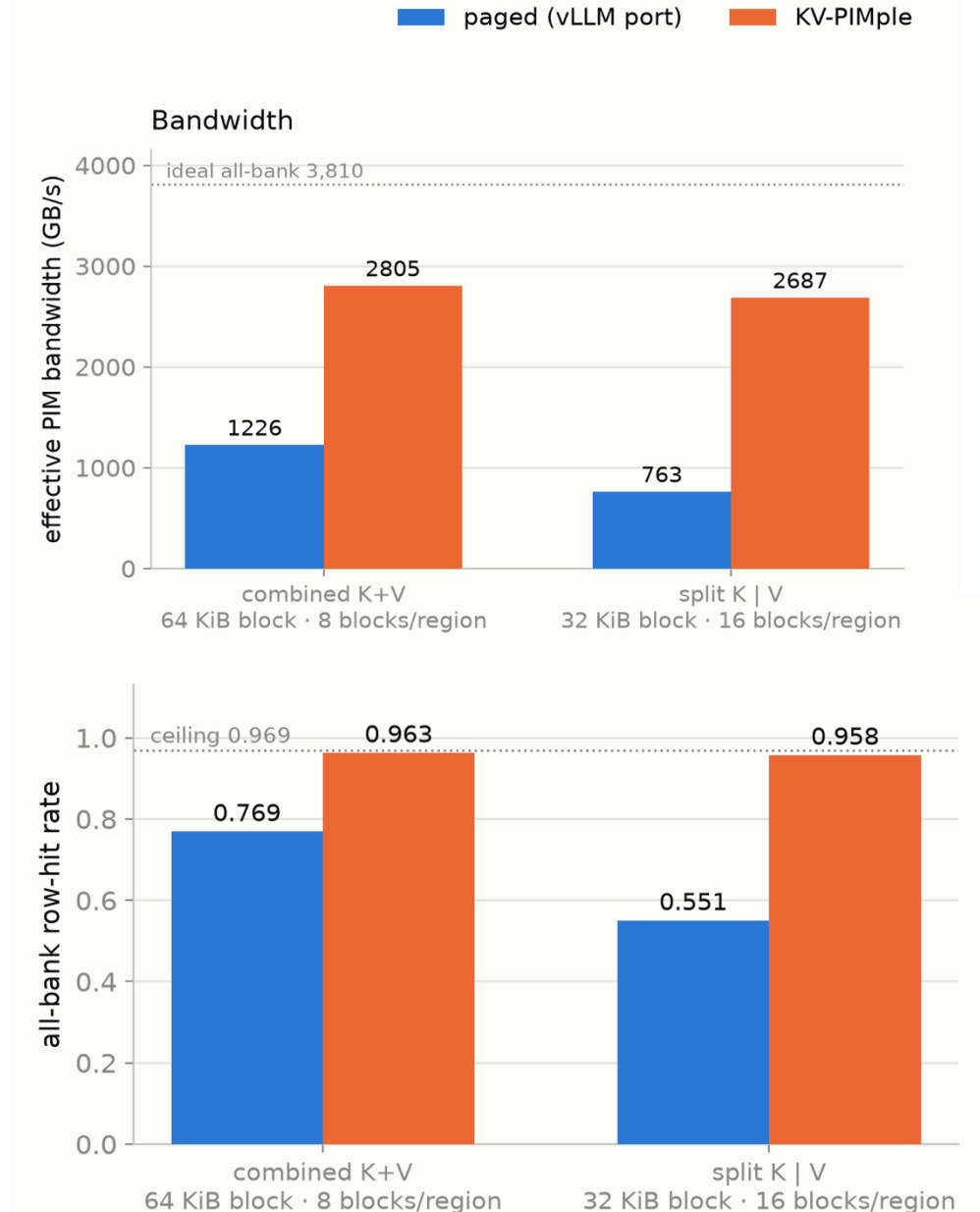
Workload:

- Row-hit rate grows with context length & block size (caps at near 0.97)
- If we just naively increase block size:
 - Internal fragmentation
 - Prefix sharing gets coarser

KV-PIMple: PIM-aware KV-cache Allocation

KV Tensor Layout

- KV-PIMple: 4 KB/token -> 64 KB block -> 8 blocks fill a row
- vLLM: K and V stay in separate tensors -> each sees half the bytes -> 32 KB block -> 16 blocks fill a row
- Smaller block per tensor -> a row activation covers half as many reads
- Paged pays one activation per block either way -> row-hit 0.77 -> 0.55
- KV-PIMple still fills a whole row before moving on -> row-hit 0.96 -> 0.96 (at the 0.969 ceiling)
- Bandwidth gain therefore widens: 2.3x -> 3.5x
- Note: Every measurement before/after this slide were run on 64 KB blocks

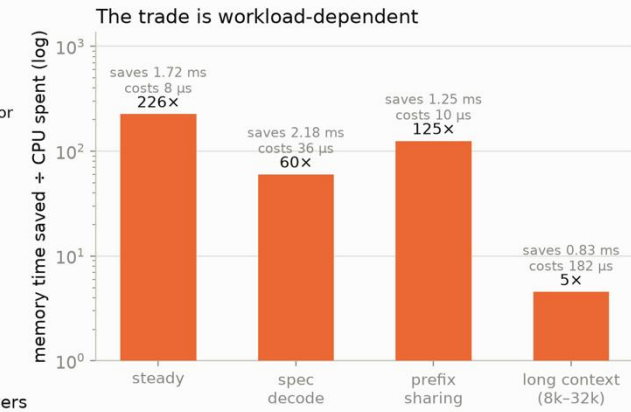
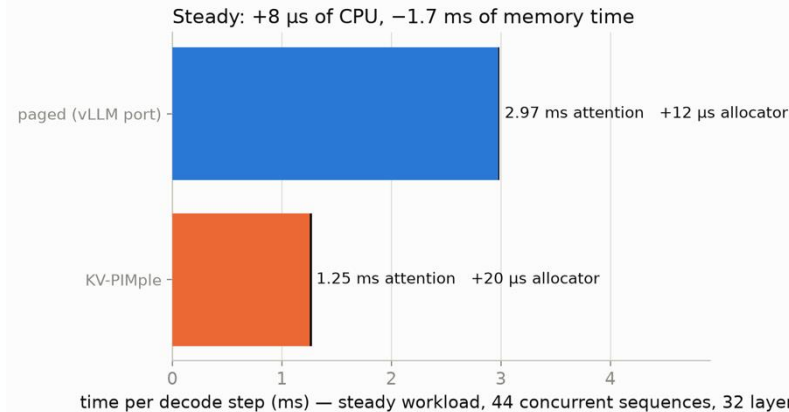
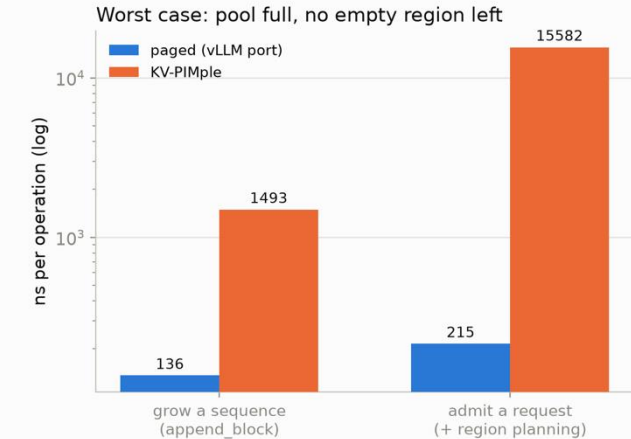
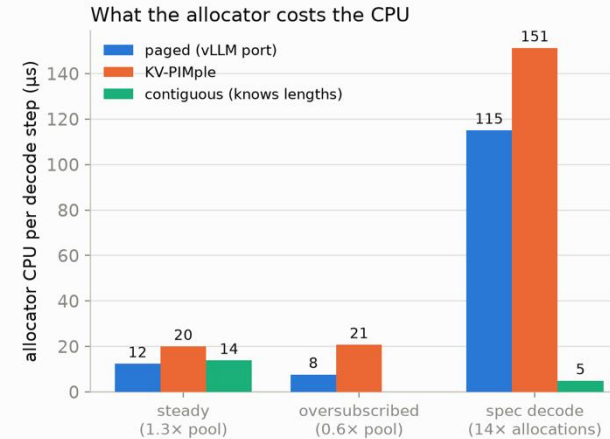


KV-PIMple: PIM-aware KV-cache Allocation

Cost of Implementations

- vLLM pops any block off a free list (one operation). We pop the lowest free slot from that sequence's region heap, and the lowest empty region when it fills -> ordered structures instead of a single list
- Headroom:** an empty region is almost always available
- Oversubscribed:** no region is ever empty -> falls back to scanning all regions
- Speculative decoding:** each round forks 4 draft branches, keeps one and frees the rest -> churn-dominated: 110 allocations/step vs. 6 in steady
- Worst case per operation:** the admission cost is best-fit region planning, which sorts every region and is **droppable** (measured as a no-op)
- Memory:** region bookkeeping is **70 KiB vs vLLM's 113 KiB** free list -> no memory cost
- The trade: Increase CPU time per decode step to gain memory time. Benefit-to-cost runs from 226x on steady down to 5x on long context

Cost of KV-PIMple allocation — measured, CPython, harness overhead (92 ns/call) subtracted



attention time = measured decode latency x 32 layers x that workload's own mean concurrency (not a fixed batch)

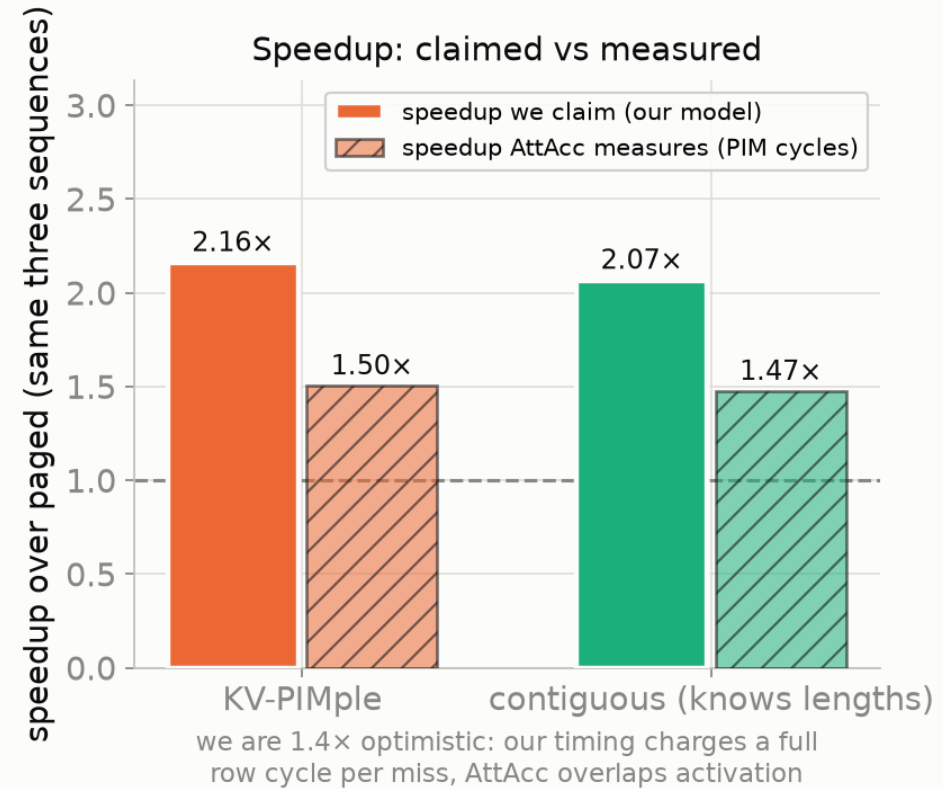
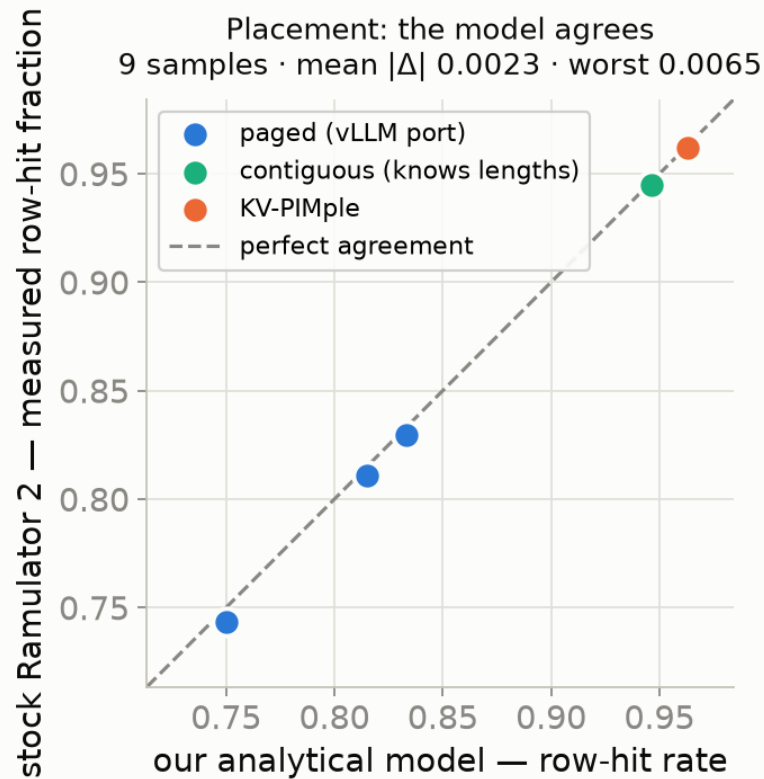
Cost and trade-offs of KV-PIMple compared to plain Paging

KV-PIMple: PIM-aware KV-cache Allocation

Validation Against Other Simulators

KV-PIMple_sim generates memory traces -> run inside Ramulator & AttAcc

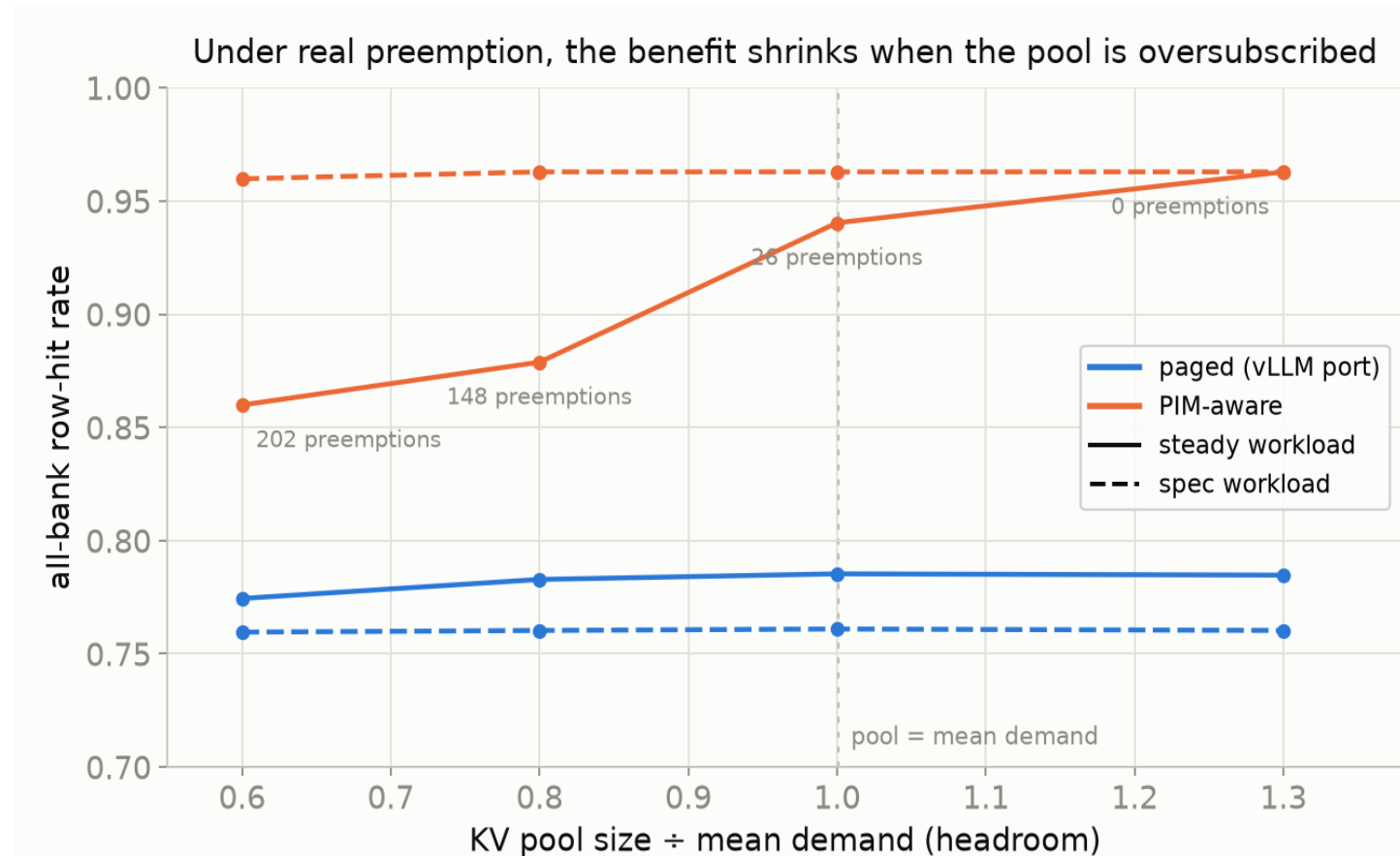
Cross-validation: placement confirmed, timing overstated by 1.4x



KV-PIMple: PIM-aware KV-cache Allocation

Gaps & Challenges

- **Preemption:** Benefits shrink when the memory pool is oversubscribed -> **2.3x** to **1.38x** -> still above the baseline
- **Long Context:** thousands of tokens per request (which is normal) then the plain Paged allocator easily reaches **95%+** row-hit rate
- **Too many heads:** $\#_heads$ is proportional to the token size -> saturate the regions -> row hit -> our advantages shrink
- Timing overstated compared to AttAcc
- **vLLM:** KV-PIMple DID NOT run vLLM directly since I can't afford the time & resources for that. KV-PIMple just modeled vLLM's allocation policy and measure 1 representative layer
- KV-PIMple only measured where the KV go, not the writes themselves



Speculative workload survives since the requests finish too fast to be evicted

KV-PIMple: PIM-aware KV-cache Allocation

Fun Fact

- KV is my initial. KV also means Key/Value
- PIMple rhymes with simple, which is the point. All I did was changing the policy on top of an already-existed allocator
- PIMple also literally means pimple, since I'm an adolescence and still have many pimples

References

- [1] Yongchan Jung, Shafayat Mowla Anik, Byeong Kil Lee, and Jeeho Ryoo. 2026. RH+: Row-Hit-Optimized Scheduling for PIM-based LLM Inference. arXiv:2606.05511 [cs.ET]. <https://arxiv.org/abs/2606.05511>
- [2] Haocong Luo, Yahya Can Tuğrul, F. Nisa Bostancı, Ataberk Olgun, A. Giray Yağlıkçı, and Onur Mutlu. 2023. Ramulator 2.0: A Modern, Modular, and Extensible DRAM Simulator. arXiv:2308.11030 [cs.AR]. <https://arxiv.org/abs/2308.11030>
- [3] SAFARI Research Group. 2023. Ramulator 2.0 (source code). <https://github.com/CMU-SAFARI/ramulator2>
- [4] Jaehyun Park, Jaewan Choi, Kwanhee Kyung, Michael Jaemin Kim, Yongsuk Kwon, Nam Sung Kim, and Jung Ho Ahn. 2024. AttAcc! Unleashing the Power of PIM for Batched Transformer-based Generative Model Inference. In *ASPLOS '24*, Vol. 2. ACM, 103–119. <https://doi.org/10.1145/3620665.3640422>
- [5] SCALE Lab, Seoul National University. 2024. AttAcc Simulator (source code). https://github.com/scale-snu/attacc_simulator
- [6] Sukhan Lee, Shin-haeng Kang, Jaehoon Lee, et al. 2021. Hardware Architecture and Software Stack for PIM Based on Commercial DRAM Technology. In *ISCA '21*. IEEE, 43–56. <https://doi.org/10.1109/ISCA52012.2021.00013>

References

- [7] vLLM Project. 2023. vLLM (source code). <https://github.com/vllm-project/vllm>
- [8] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. <https://arxiv.org/pdf/2309.06180>
- [9] Guseul Heo, Sangyeop Lee, Jaehong Cho, Hyunmin Choi, Sanghyeon Lee, Hyungkyu Ham, Gwangsun Kim, Divya Mahajan, and Jongse Park. 2024. NeuPIMs: NPU-PIM Heterogeneous Acceleration for Batched LLM Inferencing. In ASPLOS '24, Vol. 3. ACM, 722–737. <https://doi.org/10.1145/3620666.3651380>
- [10] CASYS Lab, KAIST. 2024. NeuPIMs Simulator (source code). <https://github.com/casys-kaist/NeuPIMs>
- [11] Hyucksung Kwon, Kyungmo Koo, Janghyeon Kim, et al. 2024. PIMphony: Overcoming Bandwidth and Capacity Inefficiency in PIM-based Long-Context LLM Inference System. <https://arxiv.org/abs/2412.20166>
- [12] Qingyuan Liu, Liyan Chen, Yanning Yang, Haocheng Wang, Dong Du, Zhigang Mao, Naifeng Jing, Yubin Xia, and Haibo Chen. 2025. L3: DIMM-PIM Integrated Architecture and Coordination for Scalable Long-Context LLM Inference <https://arxiv.org/abs/2504.17584>

References

- [13] Zehao Fan, Garrett Gagnon, Zhenyu Liu, and Liu Liu. 2025. Sparse Attention Remapping with Clustering for Efficient LLM Decoding on PIM. <https://arxiv.org/abs/2505.05772>
- [14] Zhichun Li, Jun Zhou, Xueqi Li, and Ninghui Sun. 2025. BlockPIM: Optimizing Memory Management for PIM-enabled Long-Context LLM Inference. In *DAC '25*. IEEE, 1–7. <https://doi.org/10.1109/dac63849.2025.11133193>
- [15] Corey Lammie, Hadjer Benmeziane, William Andrew Simon, and Irem Boybat. 2026. On Design Principles for Efficient Heterogeneous DRAM-PIM-GPU Systems. <https://arxiv.org/abs/2608.04169>
- [18] Hangyeol Kim, Sanghyun Lee, Teokkyu Suh, and Joo-Young Kim. 2026. PATTON: Enabling Commodity PIM for Production LLM Serving. <https://arxiv.org/abs/2609.11392>

Equations

Relations between row-hit rate & bandwidth

$$\text{PIM_BW} = \underbrace{\frac{N_{\text{ch}} N_{\text{bk}} S}{t_{\text{CCD}}^{\text{ab}}}}_{\text{ideal all-bank BW}} \times \underbrace{\frac{U}{N_{\text{bk}} C}}_{\text{bank participation}} \times \underbrace{\frac{t_{\text{CCD}}^{\text{ab}}}{\frac{H}{C} t_{\text{CCD}}^{\text{ab}} + \left(1 - \frac{H}{C}\right) t_{\text{RC}}}}_{\text{row-cycle stall}}$$

$$\text{PIM_BW} = \frac{N_{\text{ch}} S U}{H t_{\text{CCD}}^{\text{ab}} + (C - H) t_{\text{RC}}}$$

C = all-bank commands issued

U = bursts actually returned

N_{bk} = 16 banks/channel

$t_{\text{CCD}}^{\text{ab}}$ = 4.3 ns

H = commands whose row was already open

N_{ch} = 32 channels

S = 32 B per burst

t_{RC} = 45 ns

Token & block size calculation

$$b_{\text{tok}} = 2 n_{\text{kv}} d_{\text{head}} s_{\text{dtype}} \qquad R = N_{\text{ch}} N_{\text{bk}} W_{\text{row}}$$

$$T^* = \frac{R}{b_{\text{tok}}} = \frac{32 \times 16 \times 1 \text{ KB}}{2 n_{\text{kv}} d_{\text{head}} s_{\text{dtype}}}$$

MHA-32: 16 KB/tok

GQA-8: 4 KB/tok

MQA-1: 0.5 KB/tok

$T^* = 32$

$T^* = 128$

$T^* = 1024$

Row-hit rate ceiling

$$S = \underbrace{\frac{w_{\text{pc}} \cdot \text{BL}}{8}}_{\text{one column access}} = \frac{32 \text{ bit} \cdot 8}{8} = 32 \text{ B} \implies \underbrace{N_{\text{col}} = \frac{W_{\text{row}}}{S}}_{\text{reads per open row}} = \frac{1024 \text{ B}}{32 \text{ B}} = 32$$

$$\implies \boxed{\text{M1}_{\text{max}} = 1 - \frac{1}{N_{\text{col}}} = \frac{31}{32} = 0.969}$$

w_{pc}	32 bits	HBM3 pseudo-channel width. A 64-bit channel splits into two independent 32-bit pseudo-channels, each with its own banks and rows
BL	8	Burst Length: beats transferred per column command (HBM2's 64-bit $\times$ BL4 lands on the same 32B)
S	32B	Bytes one column command returns from one bank. For PIM, the per-bank operand of one all-bank MAC
W_{row}	1 KB	Data one ACT pulls into a bank's row buffer
N_{col}	32	Column reads servable before the row must be closed

1 compulsory miss + 31 hits per row

Thank You

